Agentic Travel Planning: Where the AI Helped, and Where It Did Not

Stefan Schneider[1] , **Fabian Missbrenner**[2]
[1]Digital Rail (DB InfraGO AG), [2]TU Berlin
stefan.sh.schneider@deutschebahn.com, mail@fabimi.net

Abstract

Many useful agentic tasks share one shape: a person states an open-ended goal in ordinary language, and the result is only useful if it satisfies hard constraints they never spelled out formally. Multi-day travel planning is a clean instance. We describe our entry to a travel-planning challenge built on this shape, and report a result we did not expect. Our system is a solver with a verifier in front of it: it never emits an itinerary the official checks have not already accepted, and every fact in a plan is read from the environment rather than generated. On released data it roughly doubles the score of the strongest baseline we could run. On the organizers' unseen queries it scored about half its local number. We diagnose that gap and find its largest identified cause is our own measurement: our constraint extraction was perfect on the one phrasing style we developed against, and missed up to a third of requirements on every other style we could test. We then measure a language model in three roles. Planning alone, it lands near zero, because grounding rather than fluency is the difficulty. Reading unfamiliar phrasing, it recovers most of what our rules miss, but cannot express values in the vocabulary the verifier compares against. Building the system, it wrote nearly all of the code. The reusable contribution is the discipline that made our own error visible.

1 Introduction

A growing class of practically valuable tasks looks like this: a human states a goal loosely, in their own words, and a system must turn it into a plan that is not merely plausible but *correct*. Correctness is not a matter of taste. A meeting cannot be booked in an occupied room; a journey cannot board a train that has already departed. What makes such tasks hard is not the language and not the optimization, but their combination: the requirements arrive fuzzy and must be satisfied exactly.

Multi-day travel planning is a good proxy for the whole class, which is why we found this challenge worth entering. The organizers describe it as a global challenge for language agents that must turn open-ended traveller intent into feasible, preference-aligned, multi-day itineraries across China [Shao *et al.*, 2024]. A request such as "three days in Suzhou for two people, we must be at the Humble Administrator's Garden between 9 and 11, keep it under 3000 yuan" has to become a concrete itinerary: real attractions with real opening hours, real restaurants, a real hotel, and every inner-city and intercity leg grounded in a sandbox database. A compositional verifier then checks each stated requirement. Requirements interact, are sometimes disjunctive, and are often implicit. Unlike slot-filling benchmarks [Xie *et al.*, 2024], a fluent answer scores nothing if a single hard requirement fails.

Scoring is a single number out of 100, dominated by the fraction of itineraries satisfying *every* stated requirement (weight 0.4); the rest covers per-requirement satisfaction, environmental validity, and three preference terms that are zeroed for any itinerary failing a requirement (Section 3).

Our system takes the benchmark's own finding — that neuro-symbolic agents beat purely neural ones by roughly an order of magnitude on constraint satisfaction — to its conclusion. We put a verifier in front of a deterministic solver and let nothing past it. No itinerary fact is generated as text. The submitted system makes zero language-model calls and zero network calls, so it needs no model, no key and no egress. We had placed second in the challenge's earlier offline stage, which shares this solver but not its trust boundary: there, ground-truth constraint programs were available locally and a submission could be assembled over weeks. Here, one command gets five hours for the whole query set and never sees a ground-truth field.

Then the interesting part. Locally the system scores about twice the best baseline we could run; on the organizers' unseen queries it scored roughly half its local number. That gap is the spine of this report.

Contributions. (i) A verifier-gated, interruption-safe planner and a scheduler that fits an unbounded search problem into a fixed shared wall clock (Section 2). (ii) Evidence that our score was bounded by what the solver could *represent*, not by how long it ran: five times the compute buys almost nothing, one missing solver move buys ten points of constraint satisfaction (Section 3). (iii) A measured diagnosis of a two-fold generalization gap, and the process error that hid it (Section 4). (iv) A language model measured as planner, constraint reader and system builder, with a division of labour that follows from the numbers (Section 5).

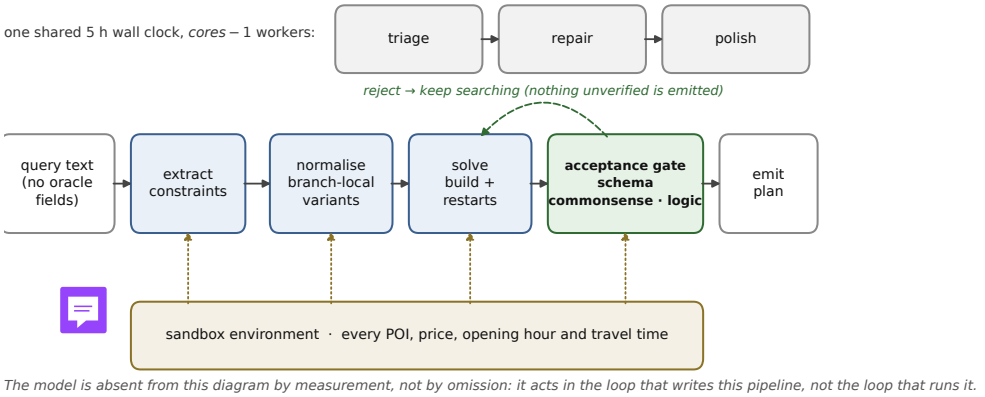


Figure 1: The system. Requirements are extracted from the query text, normalised into branch-local variants that keep disjunctions separate, and solved into a candidate itinerary. The acceptance gate is the only path to output: a rejected candidate returns to search. Every fact comes from the sandbox environment, and one scheduler spends one shared budget across the whole query set. The language model is absent from the run by measurement (Section 5), not by omission.

2 System

Contract. The system receives the visible query text and the environment tools, nothing else. Ground-truth verifier fields are stripped before parsing, and every place, price, opening hour, schedule and travel time comes from an environment query. Installation is eight packages, with no model download; the whole run is one command with no flags.

Per-query pipeline (Figure 1). *Extract* pulls requirements from the text with a rule-based grammar over the benchmark's requirement families. *Normalise* turns them into solver-facing variants, resolves named entities against the database, and keeps disjunctive requirements as separate branches end to end rather than flattening them into one conjunction. *Solve* greedily builds a base itinerary, then runs bounded randomized restarts over a small parameter space — day offsets, hotel choice, attractions per day, visit durations, transport mode, meal offsets — with an optional CP-SAT tier [Perron and Didier, 2024] and acceptance-gated repair passes. *Finalize* applies an explicit acceptance ordering: a fully accepted plan wins; failing that, a valid-but-imperfect candidate beats a higher-scoring invalid one, because the scored per-requirement term is conditional on environmental validity. This is the LLM-Modulo arrangement [Kambhampati *et al.*, 2024] with the generator replaced by a deterministic one.

The search is randomized but the system is reproducible: each restart sequence is seeded from the query identifier, so a run is bit-for-bit repeatable. Three independent five-hour runs of the same configuration produced identical scores, which is what let us later trust single-run comparisons. This was not free — an earlier version seeded restarts from a per-process-salted string hash, so runs were quietly irreproducible, and we had been attributing the resulting score spread to the solver.

Spending one clock across many queries. The run gets a single five-hour envelope for the whole query set, and every query is weighted equally, so one structurally unsolvable query must not starve the rest. Instead of giving each query an equal slice, passes run across all but one core: **triage** solves every query once at a small budget, so the set has real plans early; **repair** revisits only failing queries at a growing budget, most-nearly-passing first; **polish** improves preferences on already-passing queries with what is left — last, because preference terms are worthless while a requirement still fails. Three properties make this safe. A valid plan for every query is written *before* the first solve; every later result is kept only if it beats what is on disk; and the scheduler refuses work the envelope cannot finish. The run can be killed at any instant and still leaves a complete, non-regressed set of plans.

3 Where the points are

Because the dominant term is the fraction of itineraries satisfying *every* requirement, and because the preference terms are zeroed for any itinerary that fails one, the preference terms are bounded by the pass rate. One point of pass rate is worth about five times as much as perfecting a single preference term, so we ignored preferences almost entirely.

The search is bimodal. Per-query telemetry shows two populations and nothing between them. Queries that get solved are solved by the *first constructed plan*, in a few seconds, with zero restarts. Queries that fail spend their entire slice on dozens of randomized restarts and find nothing. Giving the failures two and a half times the budget quadrupled the restarts and flipped **one query in 68**.

So more compute was not the answer, and we measured that instead of assuming it. Across envelopes of one, three and five hours the score moves by less than a point. Every knob we had — budget, restart cap, worker count, fresh seeds — is the same intervention: draw more samples from a space of a few thousand parameter tuples. That space did not contain the fix.

One requirement family accounted for almost all of the loss. Attributing solver failures by family, the shape "be at *this named place* within *this time window*" was the *only* failing family in 45 of 61 failing queries. The sampler had no

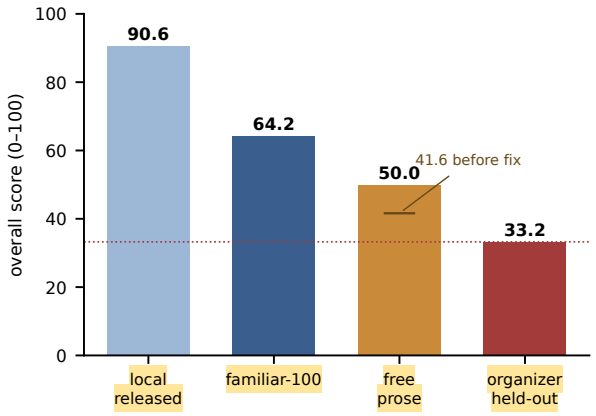


Figure 2: The same system on four query styles, strongest where requirements are enumerated in a template and weakest on the organizers' unseen set (dotted line). The marked point is free prose before the fix of Section 4. Our extraction misses 0% of stated requirements on the template style, and 12–29% on every other style we could test.

parameter that pins one activity inside a window and reflows the day around it, which is exactly why more samples never helped. Adding that single move, at identical budget, restart count and seed, raised the pass rate by ten points (Table 1). The limit was representation, not compute — a distinction worth drawing before buying more hardware.

4 The generalization gap

The organizers returned one external number for our submission, and it was about half our local score. We had predicted the local number within a few points, with a stated confidence interval, and that prediction was wrong by roughly thirty points. We report the diagnosis because it generalizes further than the system does.

What actually failed. The leaderboard later published the seven metrics behind that aggregate, and they localise the failure precisely. Environmental validity is essentially unchanged from our local runs (per-clause $99.7 \rightarrow 99.5$): every fact we emit still resolves in the database, on queries we have never seen. What collapsed is requirement satisfaction — per-requirement $90 \rightarrow 55$, and the fraction of itineraries satisfying *every* requirement $44 \rightarrow \mathbf{0}$. Grounding generalised; knowing what to ground did not. That also rules out a crashed run, since empty plans cannot score 99.5 on validity.

The exact zero is expected, not a second defect: a query passes only if all eleven of its requirements do, and a 58% per-requirement rate leaves a quarter of a query per hundred, which makes observing zero the likeliest single outcome. One consequence shaped what we did next. The pass rate carries weight 0.4 and zeroes all three preference terms, so **55% of the total weight currently returns nothing**, and it stays near zero until the per-requirement rate reaches roughly 0.8. Partial coverage fixes earn only their per-requirement share; crossing that threshold is worth several times more.

The largest identified cause is our own measurement. Our rule-based extraction has **zero** clause-level gaps on the released set we developed against — and 12–29% on every differently-phrased set we could test (Figure 2, right). The set we were clean on is the only one whose queries restate requirements as a numbered list, close to a formal specification. Ordinary prose states the same requirements in passing, and we silently dropped them. Two concrete examples, both found by reading our own code after the fact: a total budget was recognised only when the text contained the literal word "total", so "a budget of 1500 RMB" extracted nothing; and room type was recognised only in the template's wording, so "two twin rooms" extracted nothing.

This matters more than a missing feature, because an unextracted requirement is not a hard requirement — it is one we never try to satisfy *and cannot see failing*. Our scheduler decides what to repair by checking a plan against the requirements we extracted, so a requirement never extracted cannot fail: the query is marked done and promoted to polish. On free-prose queries our system reported an internal pass rate of 87% while actually scoring 7%, and finished with 86% of its budget unspent. Under extraction gaps the budget is not merely insufficient; it is spent on the wrong queries and then left on the table.

Repairing the two named causes. Both fixes are additive pattern alternatives, guarded so template phrasing is untouched; a regression test keeps verbatim copies of the prefix extractors and asserts identical output on all 100 template queries. The effect on free prose is large: overall $41.6 \rightarrow 50.0$, and restricted to the requirement clauses an English query set can actually express, the pass rate rises from 20 to 53 (Figure 2, left). On the template set the output is byte-identical, so this is generalization bought at no measured local cost. It does not close the whole gap: roughly a quarter to a half remains unexplained, most plausibly because the unseen queries are denser in requirements.

The process error, which we consider the more useful finding. We did have a held-out tripwire, and it passed. It was a 20-record fold of the *same* templated data, so it could only detect fitting the residual of one split — never fitting the phrasing of the whole artifact. Every screen, gate and ablation ran on that one style, and our extraction quality was reported throughout as a property of the parser when it was a property of the parser *on one register*. A held-out fold of the same artifact is not a held-out distribution.

5 What the language model does, and does not do

The organizers provide a model on the evaluation host, so using one was free. We ship a system that never calls it — an outcome of measurement, not a prior, and the measurements are more interesting than the decision. All three roles below use the organizers' model checkpoint, with call counts verified at the wire.

Planning alone, it is at the floor. This was the obvious experiment and we had not run it. Given the environment tools, autonomous multi-turn tool use, up to 45 steps and a prompt built from the environment's own documentation, the model plans itineraries scoring about 2 out of 100, against roughly

	Δpass rate	Δscore	verdict
Solver mechanisms (whole set, real budget)			
Window-anchored relayout	+11.0	+5.83	adopted
Window push / insert	+1.0	+0.55	adopted
Language-model roles (paired subsets)			
Reads unfamiliar phrasing	gaps −40..65%		see text
Audits our extraction	±0.0	−3.20	rejected
Repairs a rejected plan	±0.0	−0.02	rejected
Plans alone, no solver	≈ 2 of 100		floor
Upstream neuro-symbolic	≈ 9.5 of 100		baseline

Table 1: Ablations. Solver mechanisms are whole-set runs under the real budget against the preceding configuration. Language-model and baseline rows are paired subsets at reduced budget, each against a deterministic control on identical queries; they rank approaches and do not estimate performance. The last two rows score against our ≈ 48–68 on the same queries.

48–68 for our system on the same queries. The reason is specific: **not one** of its plans passed the environmental validity checks. It writes fluent itineraries containing places, times and connections that do not survive contact with the database. An upstream neuro-symbolic baseline lands near 9.5 and fails the same way for a milder reason — it grounds most facts correctly, then breaks one commonsense rule per plan. Grounding under a strict verifier, not fluency, is what this benchmark measures. Extended reasoning mode cost about 40 times more per query and timed out, so we rejected it on measured cost.

Reading unfamiliar phrasing, it does what our rules cannot — halfway. This is the role Section 4 makes newly relevant, and our earlier verdict on it was measured on the wrong data. We had screened a model-based audit of our extraction and rejected it at −3.2 score — but we screened it on the template style, where our rules are already perfect, so that experiment could not have shown a gain. Re-measured on free prose, a model that extracts requirements first cuts our extraction gaps by 40–65% and breaks nothing that already worked. Yet the score does not move, and the reason is precise: the model reliably detects *that* a requirement was stated, then emits a value outside the closed vocabulary the verifier compares against. "Hotpot" is not a cuisine label the environment knows. Detection converts into a wrong value almost one for one. So the residual difficulty is grounding a value into a fixed vocabulary — the same wall the planning experiment hit, in a smaller place. We think this is tractable and it is our next step, but we did not want to claim it before it converts.

Where it clearly did help: building the system. Nearly all of the roughly 55k lines of code we added, and its 1100 automated tests, were written by coding agents over five months and 835 commits. The human contribution was architecture, invariants, the measurement pipeline, and deciding which experiment to run next — including how to spend the budget across models, with a strong model planning the work and cheaper models implementing it under review. This is the asymmetry we would generalize: the model was most valuable in the loop that *writes* the planner, and least valuable in the loop that *runs* it. Asked instead to repair a rejected plan, it committed **zero** of 18 proposed edits — the ideas were not bad, they were not groundable.

A trap that makes such measurements lie. An early audit screen looked like +2 because a third of its calls timed out and fell back to our own parser — the "benefit" was our deterministic path with extra latency. With every call firing it became −3.2. We now require a wire-level call counter for every optional stage, and treat an arm we did not count as unmeasured.

6 Conclusion

We built a planner that cannot emit an itinerary its verifier rejected, and it beats every baseline we could run by a wide margin. It also scored about half its local number on unseen queries, and the reason was not the solver: our requirement extraction had been developed, screened and gated against a single phrasing style, with a held-out fold drawn from that same style. The correction is cheap. The lesson is not.

On the language model, our result is a division of labour rather than a verdict. Asked to plan, it scores near zero: this benchmark measures grounding under a strict verifier, not fluency. Asked to read requirements out of unfamiliar prose, it recovers most of what our rules miss — then cannot express them in the vocabulary the verifier checks, which is our next step. Asked to build the system, it wrote almost all of it. The pattern we would carry to other constrained agentic tasks: let the model handle language, let a verifier own truth, and make sure the evaluation can embarrass you before the leaderboard does.

Acknowledgements

We thank the ChinaTravel organizers for the benchmark, the sandbox environment, and their ~~prompt~~ clarifications on the execution contract.

References

[Kambhampati *et al.*, 2024] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[Perron and Didier, 2024] Laurent Perron and Frédéric Didier. CP-SAT. https://developers.google.com/optimization/cp/cp_solver, 2024. Google OR-Tools.

[Shao *et al.*, 2024] Jie-Jing Shao, Bo-Wen Zhang, Xiao-Wen Yang, Bai-Zhi Chen, Si-Yu Han, Jing-Hao Pang, Wen-Da Wei, Guohao Cai, Zhenhua Dong, Lan-Zhe Guo, and Yu-Feng Li. ChinaTravel: An open-ended benchmark for language agents in Chinese travel planning. *arXiv preprint arXiv:2412.13682*, 2024.

[Xie *et al.*, 2024] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. TravelPlanner: A benchmark for real-world planning with language agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.